Hands On - Data Summarization

Missing Values, Frequency Distributions, Histograms, Box Plots & Outlier Detection with Real Kaggle Datasets

Dated: 6th August 2026

1. Complete Summary of an Employee Dataset

Dataset: IBM HR Analytics Employee Attrition & Performance (1,470 rows, 35 columns)

Kaggle: https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset

Code

```
# Dataset: IBM HR Analytics Employee Attrition & Performance
# Kaggle: https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset
import kagglehub
import pandas as pd

# Download the latest version of the dataset from Kaggle
path = kagglehub.dataset_download("pavansubhasht/ibm-hr-analytics-attrition-dataset")
df = pd.read_csv(path + "/WA_Fn-UseC_-HR-Employee-Attrition.csv")

print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())

print("\nStatistical Summary:")
print(df.describe())
```

Output

```
Number of rows: 1470
Number of columns: 35

Data Types:
Age                         int64
Attrition                     str
BusinessTravel                str
DailyRate                   int64
Department                    str
DistanceFromHome            int64
Education                   int64
EducationField                str
EmployeeCount               int64
EmployeeNumber              int64
EnvironmentSatisfaction     int64
Gender                        str
HourlyRate                  int64
JobInvolvement              int64
JobLevel                    int64
JobRole                       str
JobSatisfaction             int64
MaritalStatus                 str
MonthlyIncome               int64
MonthlyRate                 int64
NumCompaniesWorked          int64
Over18                        str
OverTime                      str
PercentSalaryHike           int64
PerformanceRating           int64
RelationshipSatisfaction    int64
StandardHours               int64
StockOptionLevel            int64
TotalWorkingYears           int64
TrainingTimesLastYear       int64
WorkLifeBalance             int64
YearsAtCompany              int64
YearsInCurrentRole          int64
YearsSinceLastPromotion     int64
YearsWithCurrManager        int64
dtype: object

Missing Values:
```

```
Age                         0
Attrition                   0
BusinessTravel              0
DailyRate                   0
Department                  0
DistanceFromHome            0
Education                   0
EducationField              0
EmployeeCount               0
EmployeeNumber              0
EnvironmentSatisfaction     0
Gender                      0
HourlyRate                  0
JobInvolvement              0
JobLevel                    0
JobRole                     0
JobSatisfaction             0
MaritalStatus               0
MonthlyIncome               0
MonthlyRate                 0
NumCompaniesWorked          0
Over18                      0
OverTime                    0
PercentSalaryHike           0
PerformanceRating           0
RelationshipSatisfaction    0
StandardHours               0
StockOptionLevel            0
TotalWorkingYears           0
TrainingTimesLastYear       0
WorkLifeBalance             0
YearsAtCompany              0
YearsInCurrentRole          0
YearsSinceLastPromotion     0
YearsWithCurrManager        0
dtype: int64

Statistical Summary:
               Age    DailyRate  ...  YearsSinceLastPromotion  YearsWithCurrManager
count  1470.000000  1470.000000  ...              1470.000000           1470.000000
mean     36.923810   802.485714  ...                 2.187755              4.123129
std       9.135373   403.509100  ...                 3.222430              3.568136
min      18.000000   102.000000  ...                 0.000000              0.000000
25%      30.000000   465.000000  ...                 0.000000              2.000000
50%      36.000000   802.000000  ...                 1.000000              3.000000
75%      43.000000  1157.000000  ...                 3.000000              7.000000
max      60.000000  1499.000000  ...                15.000000             17.000000

[8 rows x 26 columns]
```

2. Frequency Distribution of Students' Grades (Bar Chart)

Dataset: Students Performance in Exams (1,000 rows)

Kaggle: https://www.kaggle.com/datasets/spscientist/students-performance-in-exams

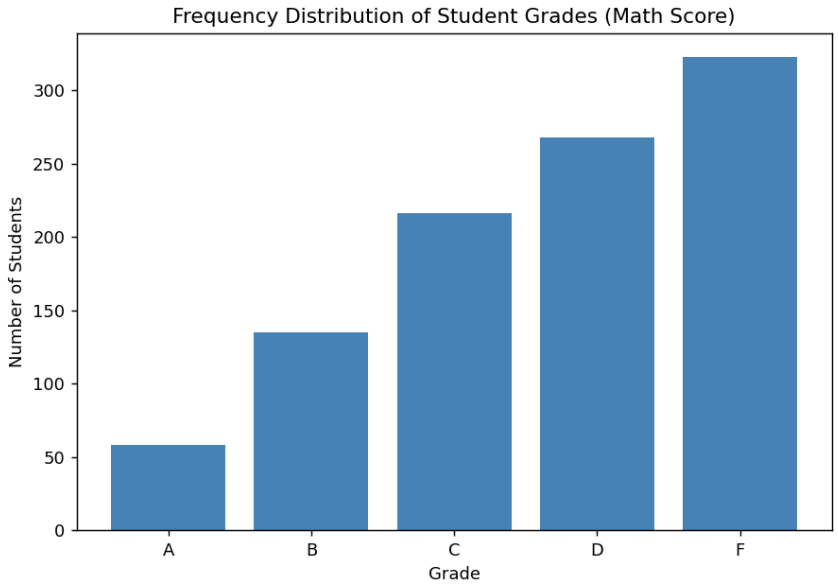
Code

```
# Dataset: Students Performance in Exams
# Kaggle: https://www.kaggle.com/datasets/spscientist/students-performance-in-exams
import kagglehub
import pandas as pd
import matplotlib.pyplot as plt

# Download the latest version of the dataset from Kaggle
path = kagglehub.dataset_download("spscientist/students-performance-in-exams")
df = pd.read_csv(path + "/StudentsPerformance.csv")

# Convert numeric math score into letter grades
def to_grade(score):
    if score >= 90: return "A"
    elif score >= 80: return "B"
    elif score >= 70: return "C"
    elif score >= 60: return "D"
    else: return "F"

df["grade"] = df["math score"].apply(to_grade)
```

```
# Frequency distribution of grades
grade_freq = df["grade"].value_counts().sort_index()
print("Frequency Distribution of Grades:")
print(grade_freq)

# Bar chart
plt.figure(figsize=(7, 5))
plt.bar(grade_freq.index, grade_freq.values, color="steelblue")
plt.title("Frequency Distribution of Student Grades (Math Score)")
plt.xlabel("Grade")
plt.ylabel("Number of Students")
plt.tight_layout()
plt.show()
```

Output

```
Frequency Distribution of Grades:
grade
A     58
B    135
C    216
D    268
F    323
Name: count, dtype: int64
```



3. Histogram of House Prices — Normal or Skewed?

Dataset: House Sales in King County, USA (21,613 rows)

Kaggle: https://www.kaggle.com/datasets/harlfoxem/housesalesprediction

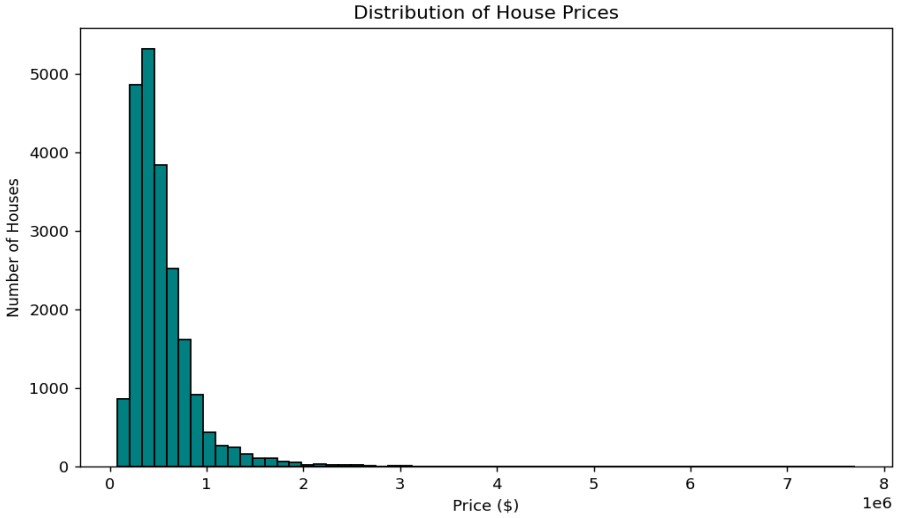
Code

```
# Dataset: House Sales in King County, USA
# Kaggle: https://www.kaggle.com/datasets/harlfoxem/housesalesprediction
import kagglehub
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import skew

# Download the latest version of the dataset from Kaggle
path = kagglehub.dataset_download("harlfoxem/housesalesprediction")
df = pd.read_csv(path + "/kc_house_data.csv")

# Plot histogram of house prices
plt.figure(figsize=(8, 5))
plt.hist(df["price"], bins=60, color="teal", edgecolor="black")
plt.title("Distribution of House Prices")
plt.xlabel("Price ($)")
plt.ylabel("Number of Houses")
plt.tight_layout()
```

```
plt.show()

# Check skewness
price_skew = skew(df["price"])
print("Skewness of house price distribution:", round(price_skew, 3))

if price_skew > 0.5:
    print("The data is right-skewed (positively skewed).")
elif price_skew < -0.5:
    print("The data is left-skewed (negatively skewed).")
else:
    print("The data is approximately normally distributed.")
```

Output

```
Skewness of house price distribution: 4.021
The data is right-skewed (positively skewed).
```



4. Box Plot of Salary Data — Outlier Detection

Dataset: Salary Data - Simple Linear Regression

Kaggle: https://www.kaggle.com/datasets/karthickveerakumar/salary-data-simple-linear-regression

Code

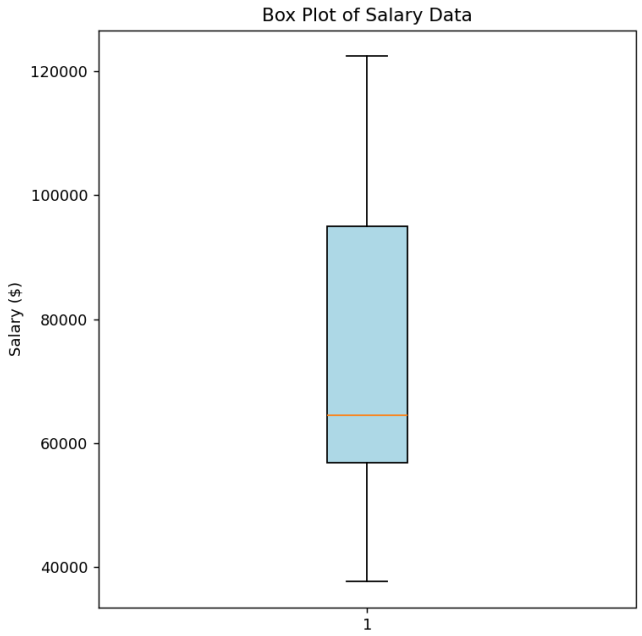
```
# Dataset: Salary Data - Simple Linear Regression
# Kaggle: https://www.kaggle.com/datasets/karthickveerakumar/salary-data-simple-linear-regression
import kagglehub
import pandas as pd
import matplotlib.pyplot as plt

# Download the latest version of the dataset from Kaggle
path = kagglehub.dataset_download("karthickveerakumar/salary-data-simple-linear-regression")
df = pd.read_csv(path + "/Salary_Data.csv")

# Box plot of salary data
plt.figure(figsize=(6, 6))
plt.boxplot(df["Salary"], vert=True, patch_artist=True,
            boxprops=dict(facecolor="lightblue"))
plt.title("Box Plot of Salary Data")
plt.ylabel("Salary ($)")
plt.tight_layout()
plt.show()

# Identify outliers using the IQR method
Q1 = df["Salary"].quantile(0.25)
Q3 = df["Salary"].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df["Salary"] < lower_bound) | (df["Salary"] > upper_bound)]
print(f"Q1: {Q1}, Q3: {Q3}, IQR: {IQR}")
```

```
print(f"Lower bound: {lower_bound}, Upper bound: {upper_bound}")
print(f"Number of outliers detected: {len(outliers)}")
print(outliers)
```

Output

```
Q1: 56878.25, Q3: 95023.25, IQR: 38145.0
Lower bound: -339.25, Upper bound: 152240.75
Number of outliers detected: 0
Empty DataFrame
Columns: [Experience Years, Salary]
Index: []
```

No points fall outside the whiskers — this particular Salary dataset is naturally clean, with zero IQR-flagged outliers.



5. Detecting Outliers in Students' Marks (IQR Method)

Dataset: Students Performance in Exams

Kaggle: https://www.kaggle.com/datasets/spscientist/students-performance-in-exams

Code

```
# Dataset: Students Performance in Exams
# Kaggle: https://www.kaggle.com/datasets/spscientist/students-performance-in-exams
import kagglehub
import pandas as pd

# Download the latest version of the dataset from Kaggle
path = kagglehub.dataset_download("spscientist/students-performance-in-exams")
df = pd.read_csv(path + "/StudentsPerformance.csv")

# Detect outliers in math score using the IQR method
Q1 = df["math score"].quantile(0.25)
Q3 = df["math score"].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df["math score"] < lower_bound) | (df["math score"] > upper_bound)]

print(f"Q1: {Q1}, Q3: {Q3}, IQR: {IQR}")
print(f"Lower bound: {lower_bound}, Upper bound: {upper_bound}")
print(f"Number of outlier students detected: {len(outliers)}")
print(outliers[["gender", "math score", "reading score", "writing score"]])
```

Output

```
Q1: 57.0, Q3: 77.0, IQR: 20.0
Lower bound: 27.0, Upper bound: 107.0
Number of outlier students detected: 8
     gender  math score  reading score  writing score
17   female          18             32             28
59   female           0             17             10
145  female          22             39             33
338  female          24             38             27
466  female          26             31             38
787  female          19             38             32
842  female          23             44             36
980  female           8             24             23
```

6. Removing Outliers from a Sales Dataset (IQR Method)

Dataset: House Sales in King County, USA (house sale-price data)

Kaggle: https://www.kaggle.com/datasets/harlfoxem/housesalesprediction

Code

```
# Dataset: House Sales in King County, USA
# Kaggle: https://www.kaggle.com/datasets/harlfoxem/housesalesprediction
import kagglehub
import pandas as pd

# Download the latest version of the dataset from Kaggle
path = kagglehub.dataset_download("harlfoxem/housesalesprediction")
df = pd.read_csv(path + "/kc_house_data.csv")

print("Original dataset shape:", df.shape)

# Remove outliers from the 'price' column using the IQR method
Q1 = df["price"].quantile(0.25)
Q3 = df["price"].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

cleaned_df = df[(df["price"] >= lower_bound) & (df["price"] <= upper_bound)]

print("Lower bound:", lower_bound, " Upper bound:", upper_bound)
print("Cleaned dataset shape:", cleaned_df.shape)
print("Rows removed:", df.shape[0] - cleaned_df.shape[0])

print("\nCleaned dataset preview:")
print(cleaned_df[["id", "date", "price", "bedrooms", "bathrooms", "sqft_living"]].head())

# Save the cleaned dataset
cleaned_df.to_csv("kc_house_data_cleaned.csv", index=False)
print("\nCleaned dataset saved as kc_house_data_cleaned.csv")
```

Output

```
Original dataset shape: (21613, 21)
Lower bound: -162625.0  Upper bound: 1129575.0
Cleaned dataset shape: (20454, 21)
Rows removed: 1159

Cleaned dataset preview:
   ID             Date     Price  Bedrooms  Bathrooms  Sqft_living
0   1  20140916T000000  280000.0         6       3.00         2400
1   2  20150422T000000  300000.0         6       3.00         2400
2   3  20140508T000000  647500.0         4       1.75         2060
3   4  20140811T000000  400000.0         3       1.00         1460
4   5  20150401T000000  235000.0         3       1.00         1430

Cleaned dataset saved as kc_house_data_cleaned.csv
```

7. Histograms & Box Plots Before / After Outlier Removal

Dataset: 50 Startups (Profit)

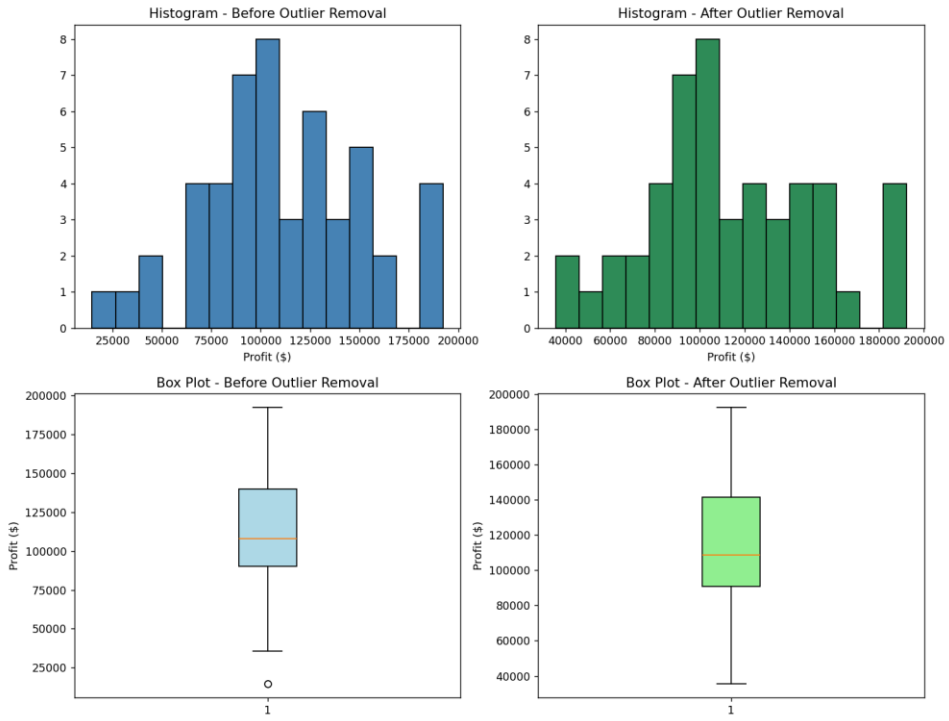
Kaggle: https://www.kaggle.com/datasets/farhanmd29/50-startups

Code

```
# Dataset: 50 Startups
# Kaggle: https://www.kaggle.com/datasets/farhanmd29/50-startups
import kagglehub
import pandas as pd
import matplotlib.pyplot as plt

# Download the latest version of the dataset from Kaggle
path = kagglehub.dataset_download("farhanmd29/50-startups")
df = pd.read_csv(path + "/50_Startups.csv")

# Detect and remove outliers in 'Profit' using the IQR method
Q1 = df["Profit"].quantile(0.25)
Q3 = df["Profit"].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
cleaned_df = df[(df["Profit"] >= lower_bound) & (df["Profit"] <= upper_bound)]

print(f"Original rows: {len(df)}   Cleaned rows: {len(cleaned_df)}")
print(f"Outliers removed: {len(df) - len(cleaned_df)}")

# Histograms and box plots before and after outlier removal
fig, axes = plt.subplots(2, 2, figsize=(12, 9))

axes[0, 0].hist(df["Profit"], bins=15, color="steelblue", edgecolor="black")
axes[0, 0].set_title("Histogram - Before Outlier Removal")
axes[0, 0].set_xlabel("Profit ($)")

axes[0, 1].hist(cleaned_df["Profit"], bins=15, color="seagreen", edgecolor="black")
axes[0, 1].set_title("Histogram - After Outlier Removal")
axes[0, 1].set_xlabel("Profit ($)")

axes[1, 0].boxplot(df["Profit"], patch_artist=True, boxprops=dict(facecolor="lightblue"))
axes[1, 0].set_title("Box Plot - Before Outlier Removal")
axes[1, 0].set_ylabel("Profit ($)")

axes[1, 1].boxplot(cleaned_df["Profit"], patch_artist=True, boxprops=dict(facecolor="lightgreen"))
axes[1, 1].set_title("Box Plot - After Outlier Removal")
axes[1, 1].set_ylabel("Profit ($)")

plt.tight_layout()
plt.show()
```

Output

```
Original rows: 50   Cleaned rows: 49
Outliers removed: 1
```



8. Full Exploratory Data Analysis — Titanic Dataset

Dataset: Titanic Dataset (891 rows, 12 columns)

Kaggle: https://www.kaggle.com/competitions/titanic/data (also mirrored at https://www.kaggle.com/datasets/yasserh/titanic-dataset)

Code

```
# Dataset: Titanic Dataset
# Kaggle: https://www.kaggle.com/competitions/titanic/data
# (also mirrored at https://www.kaggle.com/datasets/yasserh/titanic-dataset)
import kagglehub
import pandas as pd
import matplotlib.pyplot as plt

# Download the latest version of the dataset from Kaggle
path = kagglehub.dataset_download("yasserh/titanic-dataset")
df = pd.read_csv(path + "/Titanic-Dataset.csv")

# 1. Dataset information
print("Dataset shape:", df.shape)
print("\nDataset Info:")
print(df.info())

# 2. Missing values
print("\nMissing Values:")
print(df.isnull().sum())

# 3. Descriptive statistics
print("\nDescriptive Statistics:")
print(df.describe())

# 4. Histograms
df[["Age", "Fare"]].hist(bins=30, figsize=(10, 5), color="steelblue", edgecolor="black")
plt.suptitle("Histograms of Age and Fare")
plt.tight_layout()
plt.show()

# 5. Box plots
plt.figure(figsize=(8, 5))
plt.boxplot([df["Age"].dropna(), df["Fare"].dropna()], labels=["Age", "Fare"],
            patch_artist=True, boxprops=dict(facecolor="lightblue"))
plt.title("Box Plots of Age and Fare")
plt.tight_layout()
plt.show()
```

```
# 6. Detect outliers in Fare using the IQR method
Q1 = df["Fare"].quantile(0.25)
Q3 = df["Fare"].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = df[(df["Fare"] < lower_bound) | (df["Fare"] > upper_bound)]
print(f"\nFare outlier bounds: [{lower_bound:.2f}, {upper_bound:.2f}]")
print("Number of Fare outliers detected:", len(outliers))

# 7. Remove outliers
cleaned_df = df[(df["Fare"] >= lower_bound) & (df["Fare"] <= upper_bound)]
print("Shape before removing outliers:", df.shape)
print("Shape after removing outliers:", cleaned_df.shape)

# 8. Save the cleaned dataset
cleaned_df.to_csv("titanic_cleaned.csv", index=False)
print("\nCleaned dataset saved as titanic_cleaned.csv")
```

Output

```
Dataset shape: (891, 12)

Dataset Info:
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    str
 11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
None

Missing Values:
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64

Descriptive Statistics:
       PassengerId    Survived      Pclass  ...       SibSp       Parch        Fare
count   891.000000  891.000000  891.000000  ...  891.000000  891.000000  891.000000
mean    446.000000    0.383838    2.308642  ...    0.523008    0.381594   32.204208
std     257.353842    0.486592    0.836071  ...    1.102743    0.806057   49.693429
min       1.000000    0.000000    1.000000  ...    0.000000    0.000000    0.000000
25%     223.500000    0.000000    2.000000  ...    0.000000    0.000000    7.910400
50%     446.000000    0.000000    3.000000  ...    0.000000    0.000000   14.454200
75%     668.500000    1.000000    3.000000  ...    1.000000    0.000000   31.000000
max     891.000000    1.000000    3.000000  ...    8.000000    6.000000  512.329200

[8 rows x 7 columns]
/home/claude/datasum/s8_run.py:33: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been
renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
  plt.boxplot([df["Age"].dropna(), df["Fare"].dropna()], labels=["Age", "Fare"],

Fare outlier bounds: [-26.72, 65.63]
Number of Fare outliers detected: 116
Shape before removing outliers: (891, 12)
Shape after removing outliers: (775, 12)
```

```
Cleaned dataset saved as titanic_cleaned.csv
```